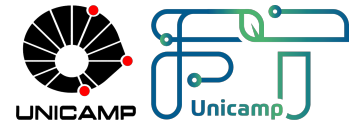


Padrões Estruturais Adapter & Decorator



Contextualização

Nesta atividade, vocês irão trabalhar com um sistema legado de cobrança corporativa.

O sistema fornecido já se encontra funcional e possui suporte à cobrança de pedidos via Boleto, Pix e Cartão de Crédito, além de ajustes aplicáveis ao valor cobrado, como descontos, juros, taxas e seguro. Entretanto, sua implementação apresenta limitações arquiteturais relacionadas à integração com gateways de pagamento externos, cujas interfaces são incompatíveis entre si, e à combinação desses ajustes de valor por meio de parâmetros booleanos.

O objetivo desta atividade não é reescrever completamente o sistema, mas analisar os problemas arquiteturais existentes e propor uma solução baseada nos padrões estruturais discutidos em aula.

Repositório base

O projeto base da atividade encontra-se disponível no repositório:

Link do repositório:

<https://github.com/vilelaricardo/padroes-estruturais-unicamp>

Descrição geral

O projeto representa um sistema corporativo responsável pela cobrança de pedidos. As formas de pagamento atualmente suportadas são:

- Boleto;
- Pix;
- Cartão de Crédito.

A cobrança via Boleto e Pix é realizada por um gateway de pagamento interno, representado pela classe `GatewayPagamentoInterno`. A cobrança via Cartão de Crédito é realizada por meio do SDK de um gateway de pagamento externo, representado pela classe `PaySecureGateway`, cuja interface, incluindo métodos, parâmetros, tipos de retorno e tratamento de erros, difere significativamente da utilizada pelo gateway interno.

Além disso, o valor final cobrado pode receber os seguintes ajustes, atualmente controlados por parâmetros booleanos no método `calcularValorFinal`:

- Desconto de fidelidade de 5%;
- Juros de parcelamento de 2,99%;
- Taxa de operação internacional de 5%;
- Seguro de transação de R\$ 4,90, em valor fixo.

Embora funcional, a implementação atual apresenta limitações arquiteturais que dificultam a evolução, a manutenção e a extensibilidade do software. Ao longo da atividade, vocês deverão analisar criticamente o código existente e propor uma solução arquitetural mais adequada utilizando padrões estruturais.

Parte 1: Refatoração com Adapter

Analise a implementação atual da integração com os gateways de pagamento e proponha uma refatoração arquitetural que reduza o acoplamento entre o serviço de cobrança e as diferentes soluções de pagamento utilizadas pelo sistema.

A solução proposta deverá:

- Definir uma abstração comum para os gateways de cobrança, a ser utilizada pelo `CobrancaService`;
- Reduzir o acoplamento do `CobrancaService` com as implementações concretas dos gateways, como `GatewayPagamentoInterno` e `PaySecureGateway`;
- Adaptar a interface do `PaySecureGateway`, que representa uma biblioteca externa, para a abstração comum definida, sem alterar o código da biblioteca externa;
- Eliminar a duplicação da lógica de seleção e conversão de gateway entre os métodos `cobrar` e `cobrarEmLote`;
- Favorecer a extensibilidade e a manutenção do sistema.

Como parte da solução proposta, a nova versão do sistema deverá incorporar suporte à seguinte forma de pagamento:

- **Carteira Digital**, integrada por meio do SDK externo `WalletPaySDK`, já disponível no pacote externo, porém ainda não utilizado pelo sistema.

Parte 2: Refatoração com Decorator

Analise a implementação atual do cálculo dos ajustes aplicados ao valor cobrado e proponha uma refatoração que substitua a lógica baseada em parâmetros booleanos por uma solução orientada a objetos.

A solução proposta deverá:

- Eliminar a dependência de parâmetros booleanos para representação dos ajustes de valor, adotando uma solução orientada a objetos que permita representar cada ajuste de forma independente e combinável;
- Permitir a combinação dos ajustes em qualquer quantidade e ordem, sem alterar a assinatura dos métodos de cobrança a cada novo ajuste criado;
- Preservar as regras e percentuais de cálculo já existentes;
- Preservar o funcionamento existente da aplicação.

Como parte da solução proposta, a nova versão do sistema deverá adicionar suporte a dois novos ajustes de valor:

- Taxa de antecipação de recebíveis de 1,5% sobre o valor;
- Taxa de emissão de nota fiscal de R\$ 2,50, em valor fixo.

Parte 3: Testes

Os testes existentes deverão continuar funcionando após a refatoração.

Além dos testes já existentes, deverão ser implementados novos cenários de teste contemplando:

- Comportamento do adapter para o `PaySecureGateway`;
- Comportamento do adapter para o novo gateway de Carteira Digital, representado pelo `WalletPaySDK`;
- Comportamento de cada novo decorator de ajuste de valor;
- Composição de múltiplos decorators em diferentes combinações e ordens.

Parte 4: Relatório Técnico

Entregar um relatório técnico contendo:

1. Análise dos principais problemas arquiteturais identificados;
2. Justificativa das decisões arquiteturais adotadas;
3. Explicação da aplicação do padrão Adapter;
4. Explicação da aplicação do padrão Decorator;
5. Impactos da refatoração sobre a organização, a manutenção e a extensibilidade do sistema.

Restrições

- Não alterar o comportamento funcional esperado do sistema;
- Não remover funcionalidades existentes;
- O sistema deve continuar compilando normalmente;
- Os testes existentes devem continuar passando após a refatoração;
- Quando necessário, os testes poderão ser adaptados às novas abstrações criadas, desde que continuem validando os mesmos cenários e comportamentos originalmente previstos;
- A atividade deve ser realizada sobre o código fornecido;
- Não substituir completamente a implementação original por um novo projeto;
- Os tipos do pacote `externo`, como `PaySecureGateway`, `WalletPaySDK` e as classes auxiliares de requisição, resposta e exceção que os acompanham, representam bibliotecas de terceiros e não devem ser modificados.

Entregáveis

Cada estudante deverá entregar:

- Relatório técnico em PDF;
- Link do repositório GitHub contendo a solução desenvolvida, informado no próprio relatório.

Observação final

O objetivo principal desta atividade não é apenas aplicar os padrões Adapter e Decorator, mas compreender os problemas arquiteturais que justificam sua utilização, avaliando criticamente seus benefícios, limitações e impactos sobre a evolução e a manutenção de sistemas reais.